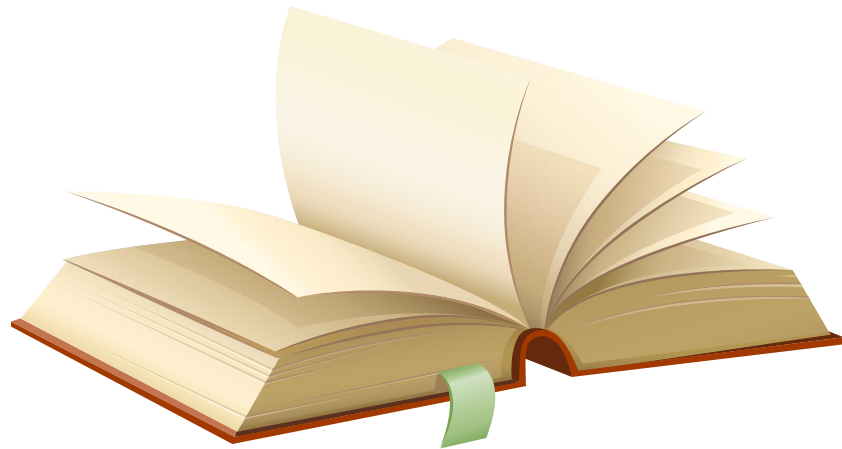


CPU漏洞实战分析

Meltdown & Spectre

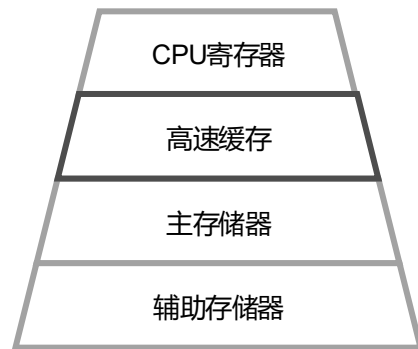


相关背景

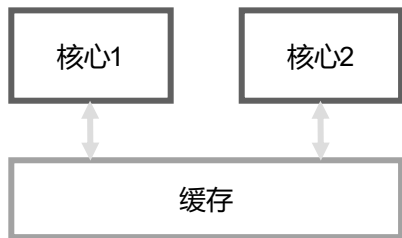
缓存 (Cache)： CPU 和主存之间数据交互的桥梁

- 存储敏感数据、密钥等信息而**不易被察觉**
 - 一旦受攻击，敏感信息易被泄露
- 不同应用的数据**共享**同一块缓存且可以互相**替换**
 - 突破安全隔离边界

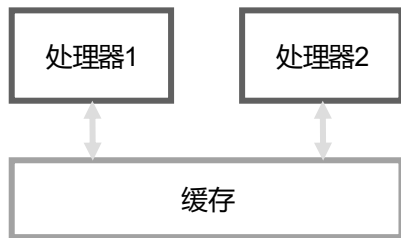
缓存侧信道攻击指攻击者利用访存时延差异推测受害者的访存行为或泄露机密信息，其具有**细粒度、高隐蔽性**等特点。针对处理器缓存的攻击很容易突破安全隔离边界实现**敏感数据泄露**，同时可以**跨平台和CPU**，影响范围广。



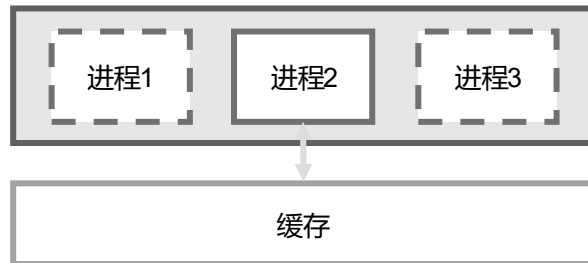
计算机存储层次结构



处理器核心间缓存共享



处理器间缓存共享



核心内进程间缓存共享

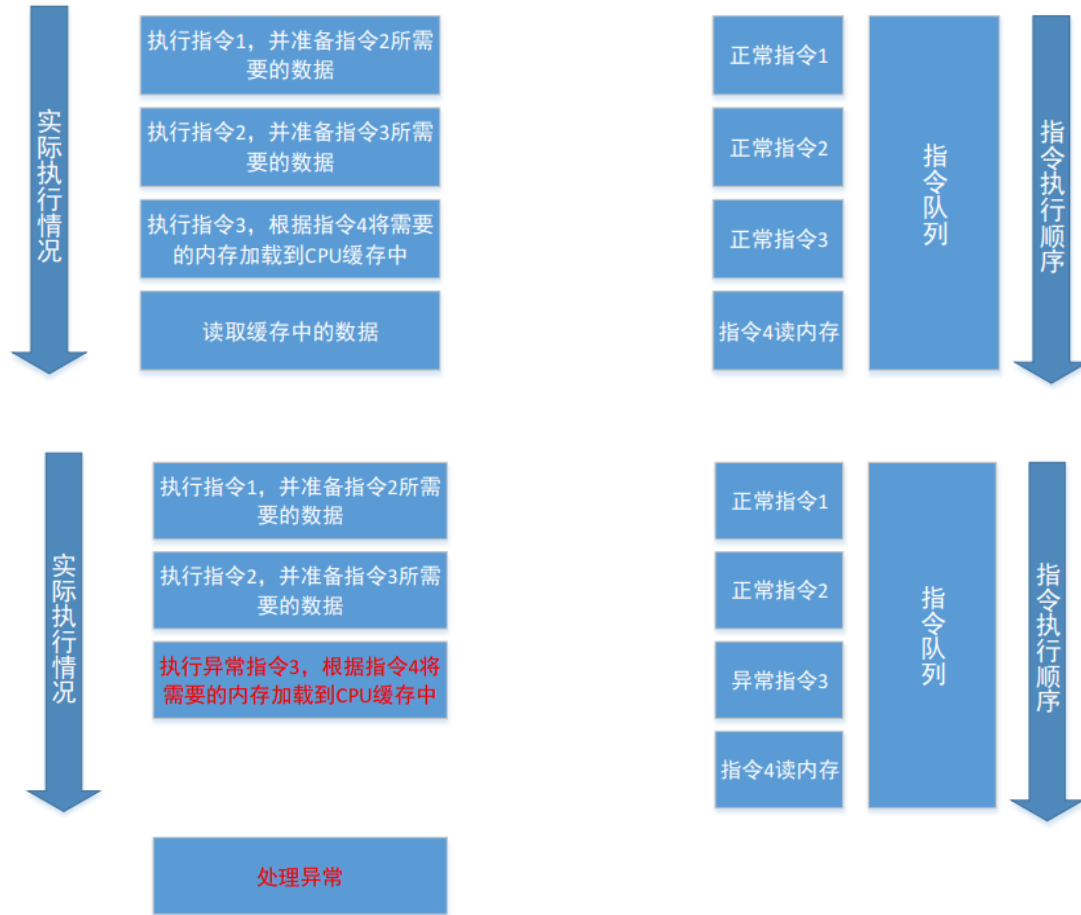
漏洞介绍

Meltdown 能够在linux、macos、windows等主流OS上，利用intel的CPU漏洞，来突破内存独立性的限制，能够通过用户程序去访问整个内核空间和其他用户其他程序的空间。其问题的根源在于利用了Intel CPU的乱序执行技术，通过对内存的响应时间差来建立一个侧信道攻击，破坏了位于用户和操作系统之间的基本隔离，“熔化”了由硬件来实现的安全边界，因此低权限用户级别的应用程序可以实现“越界”访问系统级的内存，造成数据的泄露。



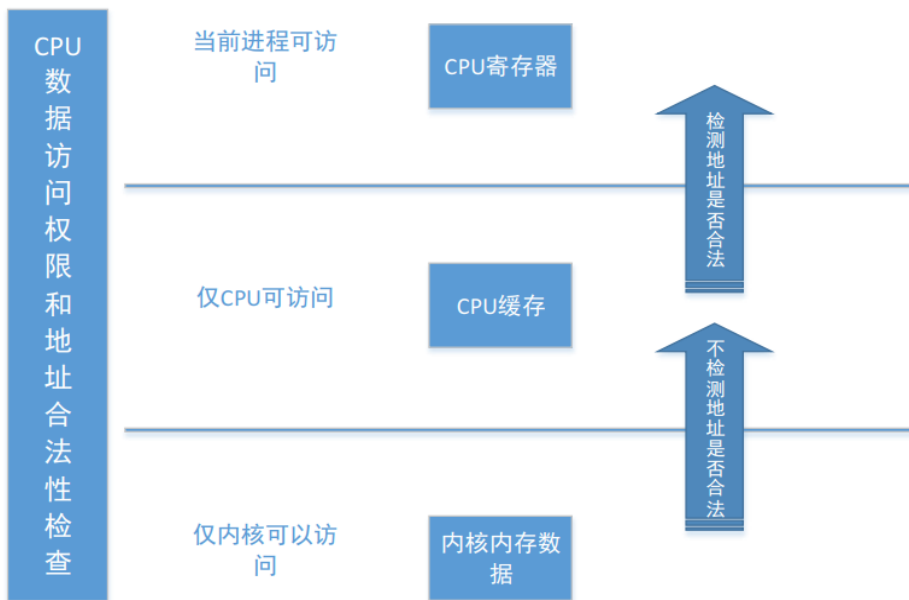
Spectre则是破坏了不同应用程序之间的隔离。问题的根源在于预测执行 (speculative execution)，这是一种优化技术，处理器会推测在未来有用的数据并执行计算。这种技术的目的在于提前准备好计算结果，当这些数据被需要时可立即使用。在此过程中，Intel没有很好地将低权限的应用程序与访问内核内存分开，这意味着攻击者可以使用恶意应用程序来获取应该被隔离的私有数据。

CPU缓存缺陷分析



对于具有预测执行能力的处理器，在实际CPU执行过程中指令4所需的内存加载环节不依赖于指令3是否能够正常执行，而且从内存到缓存加载这个环节不会验证访问的内存是否合法有效。即使指令3出现异常，指令4无法执行，但指令4所需的内存数据已加载到CPU缓存中，这一结果导致指令4即使加载的是无权限访问的内存数据，该内存数据也会加载到CPU缓存中，因为CPU是在缓存到寄存器这个环节才去检测地址是否合法，而CPU分支预测仅仅是完成内存到CPU缓存的加载，实际指令4并没有被真正的执行，因此非法访问并未触发异常的。

CPU缓存缺陷分析

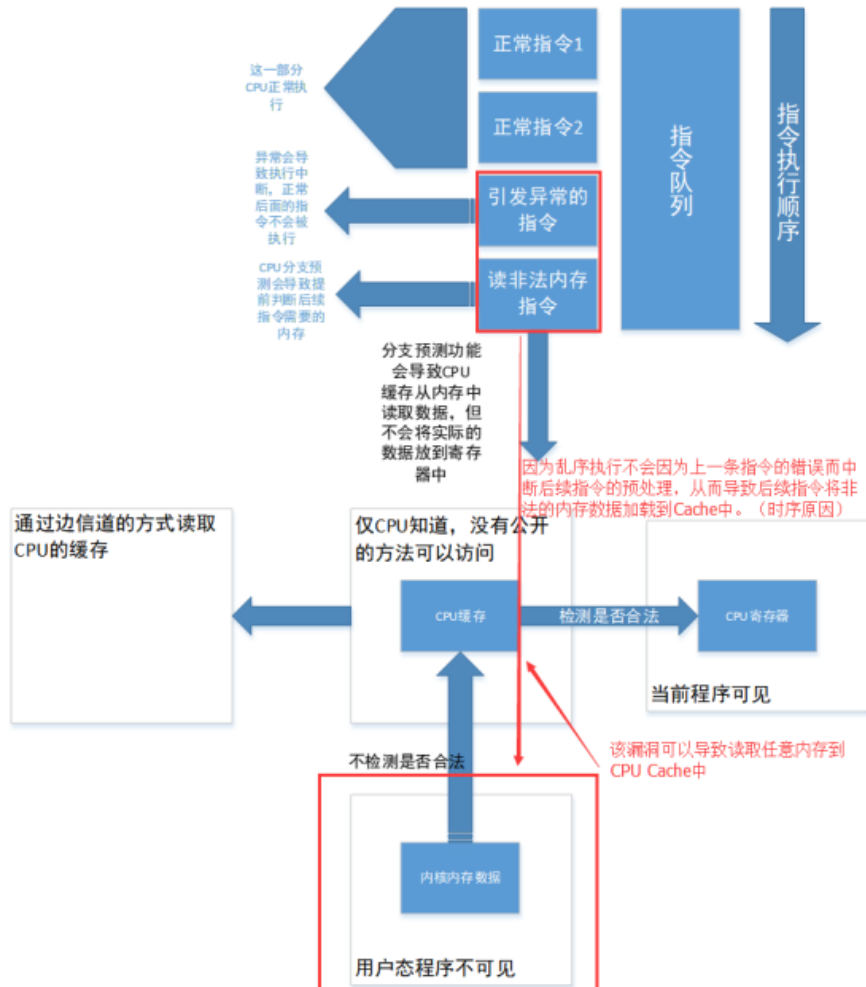


CPU缓存过程对于用户是不可访问的，只有将具体的数据放到CPU寄存器中用户才拥有访问权限，同时用户态程序也没有权限访问内核内存中的数据，因此CPU采用这种逻辑是没有问题的，但是如果有方法可以得到CPU缓存中的数据，那么这种逻辑就存在缺陷。

侧信道攻击缓存

多核之间实现cache数据共享，从而有效改善了CPU与内存访问速度之间的矛盾。而cache命中和失效对应响应时间有差别，缓存侧信道攻击正是利用CPU缓存与系统内存的读取的时间差异，从而变相猜测出CPU缓存中的数据，结合前边的缓存缺陷部分内容，产生如右图的结果。

对于CPU缓存中的数据，在用户态和内核态都是无法正常访问的，除非当CPU缓存中的数据保存到寄存器中时，会被正常的读取；除此之外，是可以通过侧信道的方式读取CPU的缓存的。





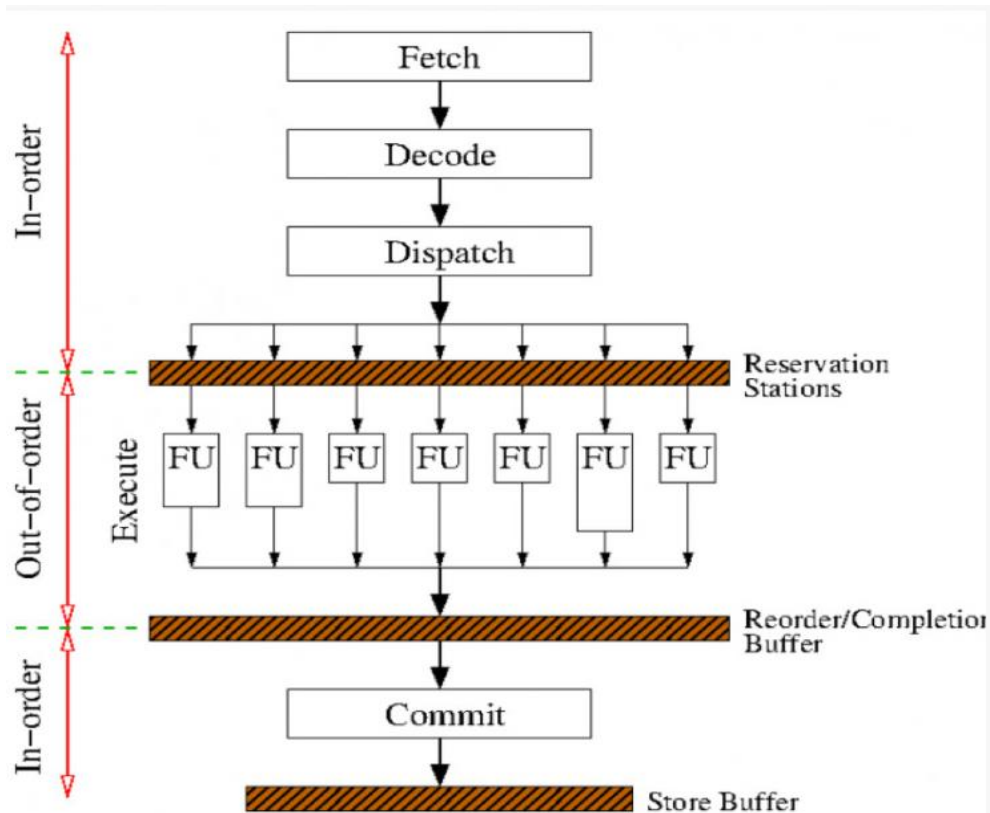
Meltdown

Meltdown 漏洞原理

Meltdown攻击利用现代CPU中乱序执行（out-of-order execution）的特性，彻底攻破原本由硬件保证的内存隔离，使得一个仅仅具有普通进程权限的攻击者可以用简单的方法来读取内核内存。

乱序执行是指当CPU中的某些指令需要等待某些资源，比如内存读取时，CPU不会在当前指令停止，而是利用空闲的计算能力继续执行后续的指令。这大大地增加了计算能力的利用率，从而提升了CPU性能。在支持乱序执行的CPU上，指令的执行并不是顺序进行的。比如后面的指令可能在前面指令执行结束之前就开始执行。然而，为了保证程序的正确性，指令retirement必须是顺序进行的，而CPU的安全检查是在指令retirement时才会进行。这样的结果是，在CPU对某一条指令进行安全检查之前，一部分在该指令后面的指令会由于CPU的乱序执行而被提前执行。

Meltdown 攻击过程



每个阶段执行的操作如下：

- 1) 获取指令，解码后存放到执行缓冲区Reservations Stations；
- 2) 乱序执行指令，结果保存在一个结果序列中；
- 3) 退休期Retired Circle，重新排列结果序列及安全检
查（如地址访问的权限检查），提交结果到寄存器。

Meltdown 攻击示例

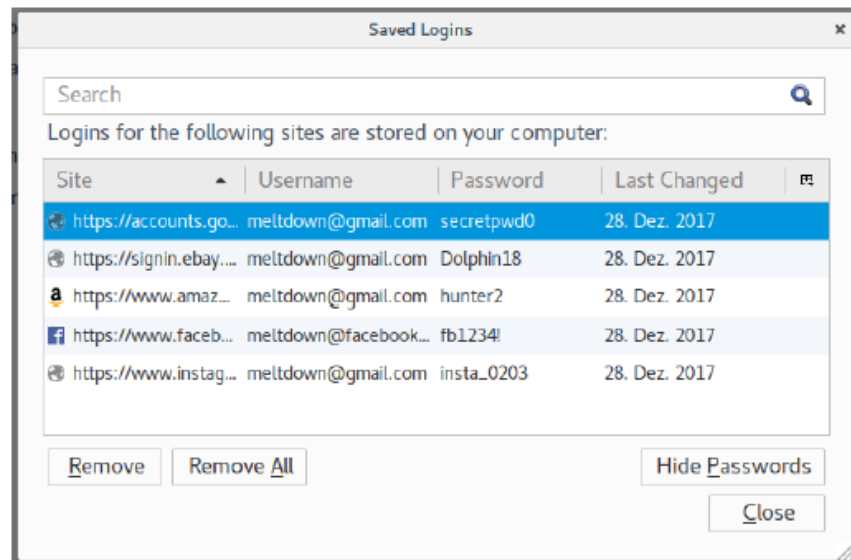
1. `;rcx = kernel address`
2. `;rbx = probe array`
3. `mov al, byte [rcx]`
4. `shl rax, 0xc` //一个内存页大小为 4KB，将 `rax` 值乘 4096，便于下一条指令实现对探测数组 `probe_array(rbx[al*4096])`
5. `mov rbx, qword [rbx + rax]` //的访问，对于不同的 `al` 值，将导致不同的内存页被访问并存放CPU 缓存中

Meltdown漏洞的利用过程有4个步骤：

- 1) 指令获取解码；
- 2) 乱序执行3条指令，指令4和指令5要等指令3中的读取内存地址的内容完成后才开始执行，指令5会将要访问的`rbx`数组元素所在的页加载到CPU Cache中；
- 3) 对2)的结果进行重新排列，对3-5条指令进行安全检测，发现访问违例，会丢弃当前执行的所有结果，恢复CPU状态到乱序执行之前的状态，但是并不会恢复CPU Cache的状态；
- 4) 通过缓存侧信道攻击，不断遍历加载`rbx[al*4096]`，由于该数据此时已经在缓存中，攻击者总会遍历出一个加载时间远小于其它的数据，推测哪个内存页被访问过了，从而推断出被访问的内核内存数据。

漏洞验证

```
f94b7690: e5 e5 e5 e5 e5 e5 e5 e5 e5 e5 e5 e5 | .....|
f94b76a0: e5 e5 e5 e5 e5 e5 e5 e5 e5 e5 e5 e5 | .....|
f94b76b0: 70 52 b8 6b 96 7f XX XX XX XX XX XX XX |pR.k.....|
f94b76c0: 09 XX XX XX XX XX XX XX XX XX XX XX XX | .....|
f94b76d0: XX XX XX XX XX XX XX XX XX XX XX XX XX | .....|
f94b76e0: XX XX XX XX XX XX XX XX XX XX XX XX 81 | .....|
f94b76f0: 12 XX e0 81 19 XX e0 81 44 6f 6c 70 68 69 6e 31 |.....Dolphin1|
f94b7700: 38 e5 e5 e5 e5 e5 e5 e5 e5 e5 e5 e5 |8.....|
f94b7710: 70 52 b8 6b 96 7f XX XX XX XX XX XX XX |pR.k.....|
f94b7720: XX XX XX XX XX XX XX XX XX XX XX XX XX | .....|
f94b7730: XX XX XX XX 4a XX XX XX XX XX XX XX XX |.....J.....|
f94b7740: XX XX XX XX XX XX XX XX XX XX XX XX XX | .....|
f94b7750: XX XX XX XX XX XX XX XX XX XX e0 81 69 6e 73 74 |.....inst|
f94b7760: 61 5f 30 32 30 33 e5 e5 e5 e5 e5 e5 e5 |a_0203.....|
f94b7770: 70 52 18 7d 28 7f XX XX XX XX XX XX XX |pR.){.....|
f94b7780: XX XX XX XX XX XX XX XX XX XX XX XX XX | .....|
f94b7790: XX XX XX XX 54 XX XX XX XX XX XX XX XX |.....T.....|
f94b77a0: XX XX XX XX XX XX XX XX XX XX XX XX XX | .....|
f94b77b0: XX XX XX XX XX XX XX XX XX XX 73 65 63 72 |.....secl|
f94b77c0: 65 74 70 77 64 30 e5 e5 e5 e5 e5 e5 e5 |etpwd0.....|
f94b77d0: 30 b4 18 7d 28 7f XX XX XX XX XX XX XX |0..){.....|
f94b77e0: XX XX XX XX XX XX XX XX XX XX XX XX XX | .....|
f94b77f0: XX XX XX XX XX XX XX XX XX XX XX XX XX | .....|
f94b7800: e5 e5 e5 e5 e5 e5 e5 e5 e5 e5 e5 e5 | .....|
f94b7810: 68 74 74 70 73 3a 2f 2f 61 64 64 6f 6e 73 2e 63 |https://addons.c/
f94b7820: 64 6e 2e 6d 6f 7a 69 6c 6c 61 2e 6e 65 74 2f 75 |dn.mozilla.net/u/
f94b7830: 73 65 72 2d 6d 65 64 69 61 2f 61 64 64 6f 6e 5f |ser-media/addon_
f94b7840: 69 63 6f 6e 73 2f 33 35 34 2f 33 35 34 33 39 39 |icons/354/354399|
f94b7850: 2d 36 34 2e 70 6e 67 3f 6d 6f 64 69 66 69 65 64 |-64.png?modified|
f94b7860: 3d 31 34 35 32 32 34 34 38 31 35 XX XX XX XX XX |1452244815.....|
```



缓解措施

KAISER：一种内核修改，无需内核映射到用户空间。

问题：存在一些限制。由于x86架构的设计，需要在用户空间中映射多个权限内存位置。这会留下Meltdown的残留攻击面，即，这些存储位置仍然可以从用户空间读取。



Spectre

Spectre 漏洞原理

Spectre攻击利用了CPU的预测执行对系统进行攻击。预测执行是另外一种CPU优化特性。在分支指令执行时，由于分支指令执行可能需要内存读取（上百个CPU周期），在分支指令执行结束之前，CPU会预测哪一个分支会被运行，提取相应的指令代码并执行，以提高CPU指令流水线的性能。

CPU的预测执行是通过分支预测单元(BPU)进行的。BPU储存了某个分支指令最近执行过的分支跳转的结果。CPU的预测执行遇到分支指令时，会根据BPU的预测结果进行跳转。当预测执行发现预测错误时，预测执行的结果将会被丢弃，CPU的状态会被重置。然而，与乱序执行类似，预测执行对CPU缓存的影响会被保留。Spectre和Meltdown攻击在这一点上比较类似。

Spectre 攻击过程

Spectre攻击主要分为三个阶段：

- 训练CPU的分支预测单元（BPU），使其在运行漏洞利用代码时会进行特定的预测执行；
- 预测执行使得CPU将要访问的地址的内容读取到CPU Cache中；
- 通过缓存测信道攻击，可以知道哪一个数组元素被访问过，也即对应的内存页存放在CPU Cache中，从而推测出地址的内容。

Spectre 攻击示例

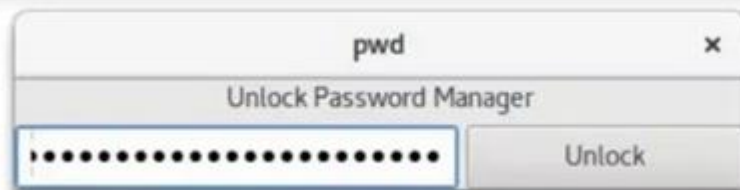
```
1.  if(x<array1_size){  
2.    y=array2[array1[x]*256];  
3.    //do something using Y that is  
4.    //observable when speculatively executed  
5.  }
```

漏洞验证

```
C:\Users\Administrator\Desktop\spectre-attack>test  
Secret is 'Test For spectre-attack!' in memory. Address:  
Reading 24 bytes:  
Reading at malicious_x = 0xfbf0... Success: 0x54='  
Reading at malicious_x = 0  
Reading at malicious_x = 0  
Reading at malicious_x = 0  
Reading at malicious_x = 0  
Reading at malicious_x = 0  
Reading at malicious_x = 0  
Reading at malicious_x = 0  
Reading at malicious_x = 0  
Reading at malicious_x = 0  
Reading at malicious_x = 0  
Reading at malicious_x = 0  
Reading at malicious_x = 0  
Reading at malicious_x = 0  
Reading at malicious_x = 0  
Reading at malicious_x = 0  
Reading at malicious_x = 0  
Reading at malicious_x = 0  
Reading at malicious_x = 0  
Reading at malicious_x = 0  
Reading at malicious_x = 0  
Reading at malicious_x = 0  
Reading at malicious_x = 0  
Testing for Spectre attack!  
Test For spectre-attack!
```

A screenshot of a Windows File Explorer window. The address bar shows the path "mschwarz@lab06:" followed by a long hexadecimal string "0xffff8803c80e8". Below the address bar, there's a search bar containing the text "this is my secr".

```
tracy@ubuntu:~/Desktop/spectre-attack$ ./test 400de8 24
Secret is 'Test For spectre-attack!' in memory, Addr is 602140.
Reading 24 bytes:
Reading at malicious_x = 0xfffffffffdef48... Success: 0x54='T' score=11
Reading at malicious_x = 0xfffffffffdef49... Success: 0x65='e' score=2
Reading at malicious_x = 0xfffffffffdef4a... Success: 0x73='s' score=2
Reading at malicious_x = 0xfffffffffdef4b... Success: 0x74='t' score=7 (second best: 0x75 score=1)
Reading at malicious_x = 0xfffffffffdef4c... Success: 0x20=' ' score=2
```



```
mschwarz@lab06:~/Documents$ taskset 0x1 ./reader 0x3c80e8040
0xfffff8803c80e8040

this is my secret password
```

```
best: 0x01 score=2)
best: 0x00 score=2)
```

```
best: 0x01 score=1)
```

```
nd best: 0x01 score=59)
```

```
best: 0x01 score=1)
```

```

d best: 0x62 score=5)

```

```
best: 0x01 score=2)
```

```
| Hmm, this does r
| eally work!
```


缓解措施

1) 序列化指令：限制预测执行。

问题：无法在所有CPU或系统配置中工作。

2) 插入推测执行阻止指令：在每个条件分支及其目标之后的指令处插入此指令。

问题：严重降低性能。

3) 微代码修复现有处理器

问题：修补程序可能会阻止推测执行或阻止推测内存读取，但这会极大地破坏性能。

Meltdown & Spectre

1) Meltdown : 依赖乱序执行

Spectre : 使用分支预测来实现推测执行

2) Meltdown : 利用了Intel处理器特有的提权漏洞，该漏洞导致推测性执行的指令可以绕过内存保护。从用户空间访问内核内存，这种访问会导致异常，但是在异常被发布之前，访问之后的代码会通过缓存信道泄漏被访问内存的内容。

Spectre : Spectre的利用难度要大很多，对分支预测的利用，无论是分析成本，还是攻击成本，都大了不少，包括 AMD、ARM、Intel等在内的大多数处理器都存在该漏洞。此外，被广泛用于缓解 Meltdown 攻击的KAISER补丁并不能防止 Spectre 攻击。



漏洞复现

实验环境

工具清单	
虚拟机	VMware Workstation11.0.0
系统镜像1	ubuntu-16.04.3-desktop-amd64.iso
内核版本1	Linux ubuntu 4.13.0-41-generic
系统镜像2	CentOS-7-x86_64-DVD-1708.iso
内核版本2	Linux localhost.localdomain 3.10.0-693.el7.x86_64
物理处理器型号	Intel (R) Core (TM) i5-7300HQ @2.5GHZ

- **CVE-2017-5753越界检查绕过（Spectre变种1）**
影响：内核和所有软件
缓解：使用修改后的编译器重新编译软件和内核，该编译器将LFENCE操作码引入到生成的代码中的适当位置
缓解的性能影响：可以忽略不计
- **CVE-2017-5715命令注入（Spectre变种2）**
影响：内核
缓解1：通过微代码更新的新操作码，应由最新的编译器使用以保护BTB（通过刷新间接分支预测器）
缓解2：将“retpoline”引入编译器，并使用它重新编译软件/操作系统
缓解的性能影响：缓解1高，缓解介质2，具体取决于CPU

漏洞检测

- **CVE-2017-5754流氓数据缓存加载（Meltdown）**
影响：内核
缓解：更新内核（使用PTI / KPTI补丁），更新内核就足够了；缓解的性能影响：低到中等
- **CVE-2018-3640流氓系统寄存器读取（变体3a）**
影响：TBC
缓解：仅限微码更新；缓解的性能影响：可以忽略不计
- **CVE-2018-3639随机存储绕过（变体4）**
影响：使用JIT的软件（没有针对内核的已知利用）
缓解：微码更新+内核更新使受影响的软件能够自我保护；缓解的性能影响：低到中等

实验环境

工具清单	
虚拟机	VMware Workstation11.0.0
系统镜像1	ubuntu-16.04.3-desktop-amd64.iso
内核版本1	Linux ubuntu 4.13.0-41-generic
系统镜像2	CentOS-7-x86_64-DVD-1708.iso
内核版本2	Linux localhost.localdomain 3.10.0-693.el7.x86_64
物理处理器型号	Intel (R) Core (TM) i5-7300HQ @2.5GHz

漏洞检测

```
Hardware check
* Hardware support (CPU microcode) for mitigation techniques
* Indirect Branch Restricted Speculation (IBRS)
  * SPEC_CTRL MSR is available: YES
  * CPU indicates IBRS capability: YES (SPEC_CTRL feature bit)
* Indirect Branch Prediction Barrier (IBPB)
  * PRED_CMD MSR is available: YES
  * CPU indicates IBPB capability: YES (SPEC_CTRL feature bit)
* Single Thread Indirect Branch Predictors (STIBP)
  * SPEC_CTRL MSR is available: YES
  * CPU indicates STIBP capability: YES (Intel STIBP feature bit)
* Speculative Store Bypass Disable (SSBD)
  * CPU indicates SSBD capability: NO
* Enhanced IBRS (IBRS_ALL)
  * CPU indicates ARCH_CAPABILITIES MSR availability: NO
  * ARCH_CAPABILITIES MSR advertises IBRS_ALL capability: NO
  * CPU explicitly indicates not being vulnerable to Meltdown (RDCL_NO): NO
  * CPU explicitly indicates not being vulnerable to Variant 4 (SSB_NO): NO
  * CPU microcode is known to cause stability problems: NO (model 0x9e family 0x6 stepping 0x9 ucode 0x84 cpuid 0x906e9)

CVE-2017-5754 [rogue data cache load] aka 'Meltdown' aka 'Variant 3'
* Mitigated according to the /sys interface: YES (Mitigation: PTI)
* Kernel supports Page Table Isolation (PTI): YES
  * PTI enabled and active: YES
  * Reduced performance impact of PTI: YES (CPU supports INVPCID, performance impact of PTI will be greatly reduced)
* Running as a Xen PV DomU: NO
> STATUS: NOT VULNERABLE (Mitigation: PTI)

CVE-2018-1014 [rogue system register read] aka 'Variant 3a'
* CPU microcode mitigates the vulnerability: NO
> STATUS: VULNERABLE (An up-to-date CPU microcode is needed to mitigate this vulnerability)
> [How to fix] The microcode of your CPU needs to be upgraded to mitigate this vulnerability. This is usually done at boot time (as done at each boot). If you're using a distro, make sure you are up to date, as microcode updates are usually shipped alongside the kernel. It depends on your CPU vendor. You can usually find out online if a microcode update is available for your CPU by searching for your CPU model. If enough, there is no additional OS, kernel or software change needed.

CVE-2018-1003 [speculative store bypass] aka 'Variant 4'
* Kernel supports speculation store bypass: NO
> STATUS: VULNERABLE (Neither your CPU nor your kernel support SSBD)
> [How to fix] Both your CPU microcode and your kernel are lacking support for mitigation. If you're using a distro kernel, upgrade the kernel from recent-enough sources. The microcode of your CPU also needs to be upgraded. This is usually done at boot time by your distro. If you're using a distro, make sure you are up to date, as microcode updates are usually shipped alongside the kernel. It depends on your CPU vendor. You can usually find out online if a microcode update is available for your CPU by searching for your CPU model. If enough, there is no additional OS, kernel or software change needed.
> STATUS: NOT VULNERABLE (Full retpoline + IBPB are mitigating the vulnerability)
```

漏洞检测

CentOS系统下检测结果

操作系统内核的差异直接影响了CPU漏洞对系统的威胁，根本上讲系统底层的相关技术决定了这类CPU漏洞的实际危害。

```
* Checking count of LFENCE instructions following a jump in kernel... NO (only 3 jump-then-lfence)
> STATUS: DISABLED (Kernel source needs to be patched to mitigate the vulnerability)

> NEW TO YOU Your kernel is too old to have the mitigation for Variant 1, you should upgrade to a newer kernel.

CVE-2017-5715 [branch target injection] aka 'Spectre Variant 2'
* Mitigation 1
  * Kernel is compiled with IBRS support: NO
  * IBRS enabled and active: NO
  * Kernel is compiled with IBPB support: NO
  * IBPB enabled and active: NO
* Mitigation 2
  * Kernel has branch predictor hardening (arm): NO
  * Kernel compiled with retpoline option: NO
  * Kernel supports RSB filling: NO
> STATUS: DISABLED (IBRS+IBPB or retpoline+IBPB+RSB filling, is needed to mitigate the vulnerability)

> NEW TO YOU To mitigate this vulnerability, you need either IBRS + IBPB, both requiring hardware support, or retpoline, which is a software mitigation. retpoline is generally preferred as it guarantees complete protection, and the performance impact is not as high as IBRS. You can take one of the two approaches.

> NEW TO YOU The microcode of your CPU needs to be upgraded to be able to use IBPB. This is usually done by the BIOS, as microcode updates are usually shipped alongside with the distro kernel. Availability of a microcode update is indicated in the Hardware Check section). An updated CPU microcode will have IBRS/IBPB capabilities indicated by the new host CPU flags to the guest. You can run this script on the host to check if the host CPU supports IBPB, such CPUs are listed with an IBRS suffix.

> NEW TO YOU Your kernel doesn't have IBPB support, so you need to either upgrade your kernel (if it's newer than 4.15.0-rc7) or use a kernel with IBPB support.

> NEW TO YOU The microcode of your CPU needs to be upgraded to be able to use IBRS. This is usually done by the BIOS, as microcode updates are usually shipped alongside with the distro kernel. Availability of a microcode update is indicated in the Hardware Check section). An updated CPU microcode will have IBRS/IBPB capabilities indicated by the new host CPU flags to the guest. You can run this script on the host to check if the host CPU supports IBRS, such CPUs are listed with an IBRS suffix.

> NEW TO YOU Your kernel doesn't have IBRS support, so you need to either upgrade your kernel (if it's newer than 4.15.0-rc7) or use a kernel with IBRS support.

CVE-2017-5754 [rogue data cache load] aka 'Meltdown' aka 'Variant 3'
* Kernel supports Page Table Isolation (PTI): NO
  * PTI enabled and active: NO
  * Reduced performance impact of PTI: YES (CPU supports INVPCID, performance impact of PTI will be reduced)
  * Running as a Xen PV DomU: NO
> STATUS: DISABLED (PTI is needed to mitigate the vulnerability)

> NEW TO YOU If you're using a distro kernel, upgrade your distro to get the latest kernel available.

CVE-2018-3640 [rogue system register read] aka 'Variant 3a'
* CPU microcode mitigates the vulnerability: NO
> STATUS: DISABLED (an up-to-date CPU microcode is needed to mitigate this vulnerability)

> NEW TO YOU The microcode of your CPU needs to be upgraded to mitigate this vulnerability. This is usually done by the BIOS, as microcode updates are usually shipped alongside with the distro kernel. Availability of a microcode update is indicated in the Hardware Check section). The microcode update is enough, there is no additional software mitigation needed.

CVE-2018-3639 [speculative store bypass] aka 'Variant 4'
* Kernel supports speculation store bypass: NO
> STATUS: DISABLED (Neither your CPU nor your kernel support SSBD)

> NEW TO YOU Both your CPU microcode and your kernel are lacking support for mitigation. If you're using a distro kernel, upgrade your distro to get the latest kernel available.
```

POC代码复现

Meltdown漏洞复现

```
[dustin@localhost meltdown-exploit]$ ./run.sh
looking for linux_proc_banner in /proc/kallsyms
cached = 39, uncached = 380, threshold 121
read ffffffff816bc080 = 25 % (score=920/1000)
read ffffffff816bc081 = 73 s (score=943/1000)
read ffffffff816bc082 = 20 (score=678/1000)
read ffffffff816bc083 = 76 v (score=939/1000)
read ffffffff816bc084 = 65 e (score=883/1000)
read ffffffff816bc085 = 72 r (score=957/1000)
read ffffffff816bc086 = 73 s (score=950/1000)
read ffffffff816bc087 = 69 i (score=876/1000)
read ffffffff816bc088 = 6f o (score=912/1000)
read ffffffff816bc089 = 6e n (score=852/1000)
read ffffffff816bc08a = 20 (score=825/1000)
read ffffffff816bc08b = 25 % (score=942/1000)
read ffffffff816bc08c = 73 s (score=961/1000)
read ffffffff816bc08d = 20 (score=775/1000)
read ffffffff816bc08e = 28 (score=871/1000)
read ffffffff816bc08f = 62 b (score=950/1000)
VULNERABLE
PLEASE POST THIS TO https://github.com/paboldin/meltdown-exploit/issues/19
VULNERABLE ON
3.10.0-693.el7.x86_64 #1 SMP Tue Aug 22 21:09:27 UTC 2017 x86_64
processor      : 0
vendor_id     : GenuineIntel
cpu family    : 6
model         : 158
model name    : Intel(R) Core(TM) i5-7300HQ CPU @ 2.50GHz
stepping      : 9
microcode     : 0x70
cpu MHz       : 2495.052
cache size    : 6144 KB
physical id    : 0
```

CentOS系统下POC代码执行结果，POC成功的读到了内核在linux_proc_banner地址上的数据。

```
[dustin@localhost SpectrePoC]$ ./spectre.out
Version: commit c712f907f03bc7f8f2b4cb6dc6a52b729c38ba9b
Using a cache hit threshold of 80.
Build: RDTSCP_SUPPORTED MFENCE_SUPPORTED CLFLUSH_SUPPORTED INTEL_MITIGATION_DISABLED LINUX_KERNEL_MITIGATION_DISABLED
Reading 40 bytes:
Reading at malicious x = 0xfffffffffffffec40... Success: 0x54='T' score=95 (second best: 0x01='?' score=45)
Reading at malicious x = 0xfffffffffffffec41... Success: 0x68='h' score=501 (second best: 0x01='?' score=278)
Reading at malicious x = 0xfffffffffffffec42... Success: 0x65='e' score=2
Reading at malicious x = 0xfffffffffffffec43... Success: 0x20=' ' score=31 (second best: 0x00='?' score=12)
Reading at malicious x = 0xfffffffffffffec44... Success: 0x40='M' score=7
Reading at malicious x = 0xfffffffffffffec45... Success: 0x61='a' score=15 (second best: 0x00='?' score=4)
Reading at malicious x = 0xfffffffffffffec46... Success: 0x67='g' score=17 (second best: 0x00='?' score=7)
Reading at malicious x = 0xfffffffffffffec47... Unclear: 0x69='i' score=990 (second best: 0x6A='j' score=564)
Reading at malicious x = 0xfffffffffffffec48... Success: 0x63='c' score=501 (second best: 0x01='?' score=248)
Reading at malicious x = 0xfffffffffffffec49... Success: 0x20=' ' score=13 (second best: 0x00='?' score=5)
Reading at malicious x = 0xfffffffffffffec4a... Success: 0x57='W' score=65 (second best: 0x00='?' score=31)
Reading at malicious x = 0xfffffffffffffec4b... Success: 0x6F='o' score=13 (second best: 0x00='?' score=5)
Reading at malicious x = 0xfffffffffffffec4c... Success: 0x72='r' score=2
Reading at malicious x = 0xfffffffffffffec4d... Success: 0x64='d' score=13 (second best: 0x00='?' score=5)
Reading at malicious x = 0xfffffffffffffec4e... Success: 0x73='s' score=19 (second best: 0x00='?' score=6)
Reading at malicious x = 0xfffffffffffffec4f... Success: 0x20=' ' score=135 (second best: 0x00='?' score=64)
Reading at malicious x = 0xfffffffffffffec50... Success: 0x61='a' score=27 (second best: 0x00='?' score=10)
Reading at malicious x = 0xfffffffffffffec51... Success: 0x72='r' score=45 (second best: 0x00='?' score=21)
Reading at malicious x = 0xfffffffffffffec52... Success: 0x65='e' score=543 (second best: 0x01='?' score=269)
Reading at malicious x = 0xfffffffffffffec53... Success: 0x20=' ' score=7
Reading at malicious x = 0xfffffffffffffec54... Success: 0x53='S' score=2
Reading at malicious x = 0xfffffffffffffec55... Success: 0x71='q' score=35 (second best: 0x00='?' score=14)
Reading at malicious x = 0xfffffffffffffec56... Success: 0x75='u' score=7
Reading at malicious x = 0xfffffffffffffec57... Success: 0x65='e' score=9 (second best: 0x00='?' score=3)
Reading at malicious x = 0xfffffffffffffec58... Success: 0x61='a' score=47 (second best: 0x62='b' score=21)
Reading at malicious x = 0xfffffffffffffec59... Success: 0x60='m' score=97 (second best: 0x00='?' score=47)
Reading at malicious x = 0xfffffffffffffec5a... Success: 0x69='i' score=2
Reading at malicious x = 0xfffffffffffffec5b... Success: 0x73='s' score=2
Reading at malicious x = 0xfffffffffffffec5c... Success: 0x68='h' score=21 (second best: 0x00='?' score=9)
Reading at malicious x = 0xfffffffffffffec5d... Success: 0x20=' ' score=31 (second best: 0x00='?' score=12)
Reading at malicious x = 0xfffffffffffffec5e... Success: 0x4F='O' score=9 (second best: 0x00='?' score=3)
Reading at malicious x = 0xfffffffffffffec5f... Success: 0x73='s' score=23 (second best: 0x00='?' score=8)
Reading at malicious x = 0xfffffffffffffec60... Success: 0x73='s' score=17 (second best: 0x00='?' score=7)
Reading at malicious x = 0xfffffffffffffec61... Success: 0x69='i' score=23 (second best: 0x00='?' score=8)
Reading at malicious x = 0xfffffffffffffec62... Success: 0x66='f' score=41 (second best: 0x00='?' score=19)
Reading at malicious x = 0xfffffffffffffec63... Success: 0x72='r' score=57 (second best: 0x00='?' score=27)
Reading at malicious x = 0xfffffffffffffec64... Success: 0x61='a' score=7
Reading at malicious x = 0xfffffffffffffec65... Success: 0x67='g' score=2
Reading at malicious x = 0xfffffffffffffec66... Success: 0x65='e' score=37 (second best: 0x01='?' score=16)
Reading at malicious x = 0xfffffffffffffec67... Success: 0x2E='.' score=27 (second best: 0x00='?' score=10)
```

在CentOS系统下，该proc攻击利用了CPU的预测执行导致了提前把私有数据放到了cpu缓存中，成功读取内存中的字符数据 “The Magic Words are Squeamish Ossifrage.”

攻击结果

在实际的攻击场景中，攻击者利用这些漏洞在一定条件下可以做到：

- 泄露出本地操作系统底层运作信息，密钥信息等；
- 通过获取泄露的信息，成功绕过内核、虚拟机超级管理器HyperVisor的隔离防护；
- 云服务中，泄露其它租户隐私信息；
- 通过浏览器泄露受害者的帐号，密码，内容，邮箱，cookie等用户隐私信息。

其他新场景下的攻击

基于硬件特性的攻击

有别于以往利用缓存共性的攻击，许多新型缓存攻击利用了处理器的硬件特性，例如查找逻辑、替换策略、预取技术、瞬态执行等。这类攻击需要攻击者对硬件架构有深入了解，因此发现的难度较大。同时，因为这类攻击利用的微架构通常对软件透明，也更难被防御机制检测。

安全机制场景下的攻击

随着安全问题受到重视，越来越多的软硬件安全机制在计算机中部署，其中就包括可信执行环境。然而，一些商用TEE在缓存侧信道攻击面前并不可信。TEE能被缓存侧信道攻击的根本原因在于不同安全域之间的缓存共享，即在缓存层面上并未做到真正的隔离。

性能驱动下的攻击

很多场景为攻击者带来了苛刻的攻击环境，例如被攻击程序的运行窗口过短、系统并发引入的噪声、缓存预取造成的干扰等。这样的场景对攻击的性能有较高的要求。缓存攻击可提升的性能主要分为三个方面：攻击的速度、攻击的精度、攻击泄露的信息量。

攻击分布情况

各类攻击在商用处理器中的分布情况

硬件平台	访问驱动下的攻击	基于建模分析的潜在攻击	针对LLC的攻击			新场景下的攻击		
			利用包含性的攻击	利用目录的攻击	利用一致性协议的攻击	针对硬件性的攻击	安全机制下的攻击	性能驱动下的攻击
Intel	√	√	√	√	√	√	√	√
AMD	√	√			√	√	√	√
ARM	√		√		√		√	√

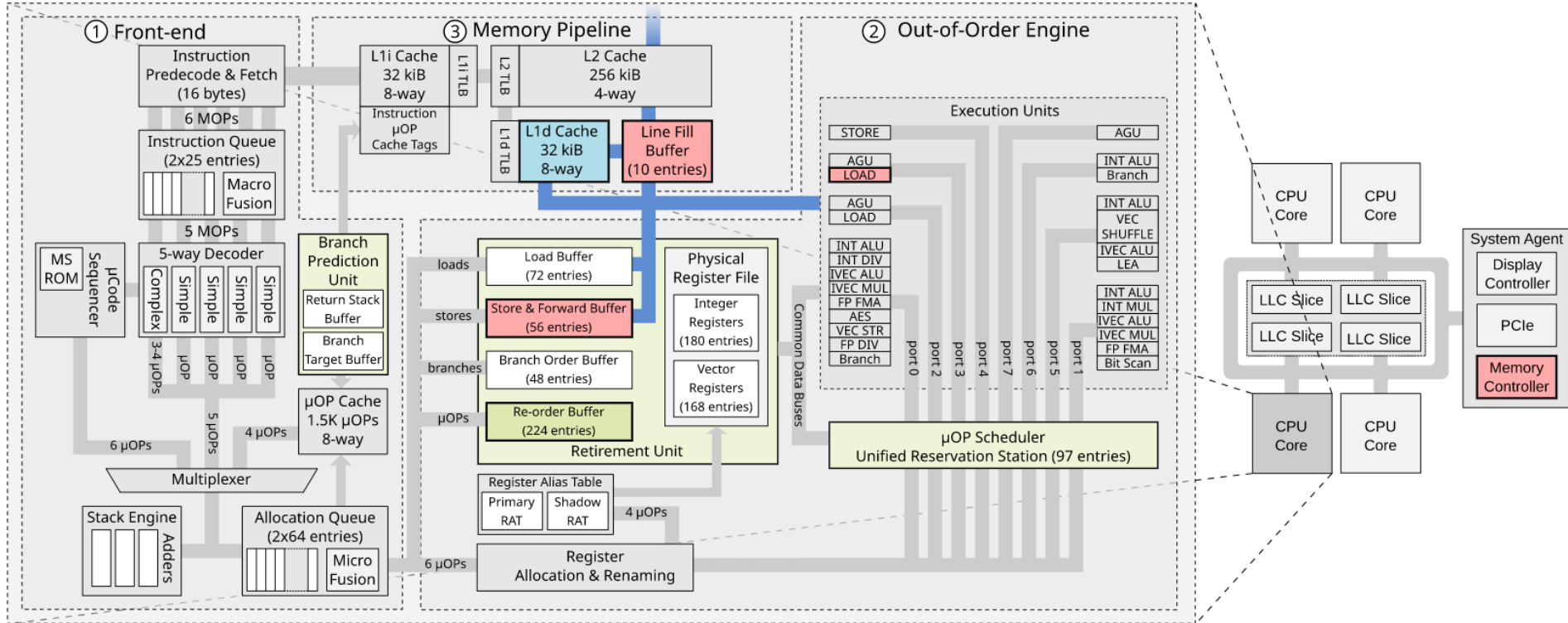
一些攻击是基于缓存共有性质的，因此具备通用性。一些新型攻击特定于缓存的具体实现，因此攻击范围较小，但其危害不容忽视。**目前主流商用处理器均未能完全抵御缓存侧信道攻击**，尤其是 Intel 处理器因为性能优化而忽略了很多安全考虑，导致几乎任何 Cache 攻击均能实现。



Cache安全增强

<http://mdsattacks.com/>

diagram



Cache安全增强方案

商用处理器Cache安全增强方案

攻击类型	Intel采用的防御方法	ARM采用的防御方法
边界检查绕过	内存屏障指令：LFENCE	新内存屏障指令：CSDB
分支目标注入	返回跳板（retpoline）	无通用缓解方法
数据缓存恶意加载	内核页表隔离技术（KPTI）	
系统寄存器恶意加载	更新微码	非必要
推测存储绕过	LFENCE指令；特殊模块寄存器（MSR）SSBD标识位	两种新内存屏障指令：SSBB, PSSBB

目前来看，商用处理器中只针对已经披露的 Meltdown 和 Spectre 攻击实现防御，**而对其他类型的缓存侧信道攻击尚未采用具体的防御手段**，随着各种攻击造成的敏感数据泄露等事件频繁发生，相应的防御机制的研究需要进一步加强。

Cache安全增强方案

针对Cache攻击的防御方案分析——基于软件的防御机制

漏洞检测方法

通过静态或动态**程序分析**找出程序中可能存在缓存侧信道漏洞的部分

用户级防御措施

在漏洞检测的基础上**增强用户程序**，常见思路包括保持特定运算的时间恒定、清除缓存等

系统级防御措施

从**操作系统/虚拟机管理程序的角度**进行防御，通过完善内存管理或锁定缓存行等方法避免攻击者恶意探查

优点：软件实现较为**灵活**，可快速部署到实际运行环境

缺点：由软件实现的缓存侧信道防御方法在一定程度上**依赖于下层微架构的实现**，或者需要硬件提供额外的支持以降低**开销**，在不同场景下效用可能有所不同，因此长期来看需要从硬件方面对这类攻击进行防范。

Cache安全增强方案

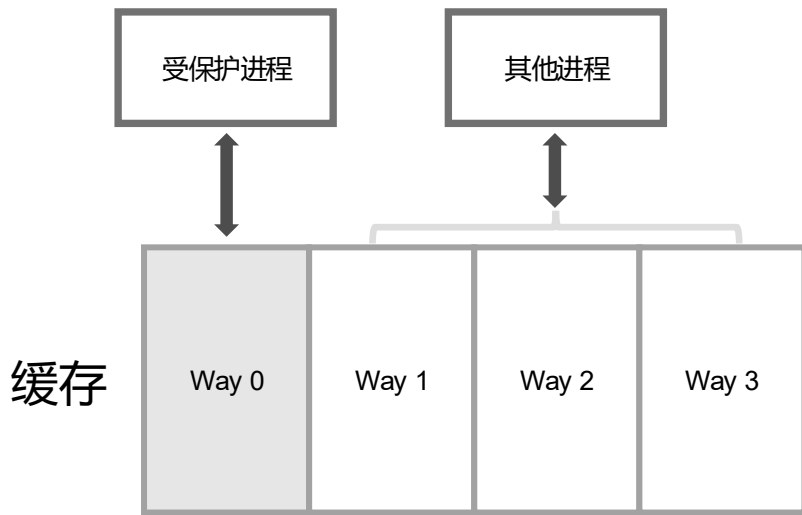
针对Cache攻击的防御方案分析——基于硬件的防御机制

缓存分组方法

将缓存**划分为不同区域**
将各区域分配给不同进程使用，真正实现隔离

优点：从原理上**完全避免**侧信道攻击

缺点：实现上受**硬件限制**，对**性能影响**也较大



基于路的缓存分组方法示意

Cache安全增强方案

针对Cache攻击的防御方案分析——基于硬件的防御机制

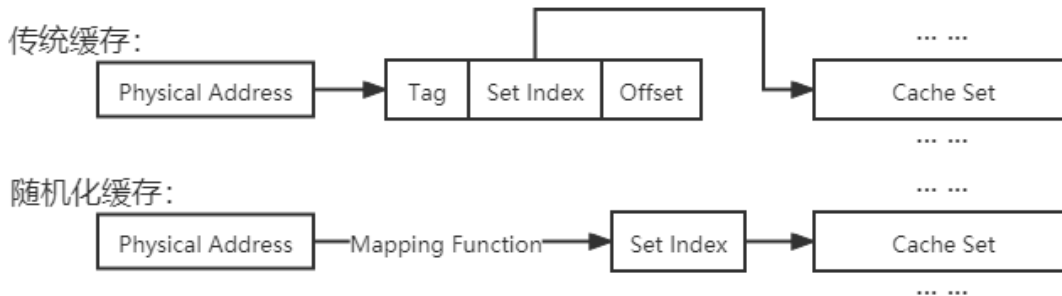
缓存随机化方法

消除特定地址在缓存中的**固定映射**，使攻击者难以推断受害者的访存地址

- **基于表**的映射方案：将随机映射存储于表，通过查表获取实际缓存索引
- **基于计算**的映射方案：根据地址通过加密等方法计算随机映射

优点：性能**开销较小**，**有效性**得到较充分的验证

缺点：**无法完全避免**侧信道攻击，但能大幅缓解



随机化缓存索引示意图

Cache安全增强方案

针对Cache攻击的防御方案分析——其他防御策略

针对目录结构的防御方法

专门防御针对目录结构的攻击
两种解决思路：

从源头出发

- 增加目录项的数量，减少目录项使用冲突

从最终目标出发

- 检测跨核心缓存行替换，重载被替换的缓存行至L2

完善软硬件协作关系

重新思考软硬件协作关系，
定义**新指令集体系结构**

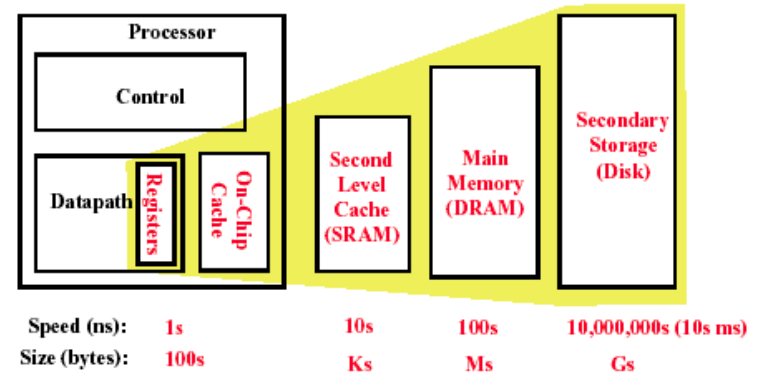
常见思路：

- 通过数据遗忘编程（Data Oblivious）消除机密信息与分支执行时延差异的关联
- 通过优化基本块执行减少指令执行时的分支预测

Summary

1. Trend and Memory Hierarchy

- Processor-Memory Performance Gap
- Cache Locality
 - Temporal locality
 - Spatial locality
- Cache hit/miss



2. Four Questions for Cache Designers

3. CPU vulnerabilities & Cache security

4. Cache performance

